

Important Python Code for Data Analytics



Python
for
Data Analytics

1 Install and Import Libraries

Install required libraries and import them in your script or notebook.

```
# Install libraries (option 1: in terminal)
pip install pandas numpy matplotlib seaborn scipy

# Install libraries (option 2: in Jupyter/Colab)
!pip install pandas numpy matplotlib seaborn scipy

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Use pip install in terminal or !pip install in Jupyter/Colab.

2 Read and Explore Data

Load a dataset and quickly check its structure.

```
# Read a CSV file
df = pd.read_csv('data.csv')

# View first few rows
df.head()

# Check shape (rows, columns)
df.shape

# Get column information
df.info()

# Summary statistics
df.describe()
```

Use these commands to understand your data quickly.

3 Data Selection and Filtering

Select specific columns, rows and filter data efficiently.

```
# Select a single column
df['age']

# Select multiple columns
df[['name', 'age', 'salary']]

# Filter rows (age > 30)
df[df['age'] > 30]

# Filter with multiple conditions
df[(df['age'] > 30) & (df['city'] == 'Bangalore')]
```

Helps you work with specific data easily.

4 Handle Missing Values

Check and handle missing (null) values.

```
# Check missing values
df.isnull().sum()

# Drop rows with missing values
df.dropna()

# Fill missing values (with mean) - recommended
df['age'] = df['age'].fillna(df['age'].mean())

# Fill missing values (with a specific value)
df['city'] = df['city'].fillna('Unknown')
```

Clean data is important for accurate analysis.

5 Data Cleaning and Transformation

Perform common data cleaning and transformation tasks.

```
# Remove duplicate rows
df.drop_duplicates(inplace=True)

# Rename columns
df.rename(columns={'old_name': 'new_name'}, inplace=True)

# Convert data type
df['date'] = pd.to_datetime(df['date'])
df['age'] = df['age'].astype(int)

# Create a new column
df['age_group'] = np.where(df['age'] >= 30, '30+', '<30')
```

Prepare data for analysis by cleaning and transforming it.

6 Grouping and Aggregation

Summarize data using groupby and aggregation functions.

```
# Group by a column and find mean
df.groupby('city')['salary'].mean()

# Multiple aggregations
df.groupby('department').agg({
    'salary': 'mean',
    'age': 'count'
})

# Pivot table (quick summary)
pd.pivot_table(df, values='salary', index='city',
               columns='gender', aggfunc='mean')
```

Useful for finding trends and patterns in data.

7 Data Visualization

Create common plots to visualize data.

```
# Line plot (ensure date is datetime)
df['date'] = pd.to_datetime(df['date'])
plt.plot(df['date'], df['sales'])
plt.xlabel('Date')
plt.ylabel('Sales')
plt.title('Sales Over Time')
plt.show()

# Histogram
sns.histplot(df['salary'], kde=True)
plt.show()

# Bar plot
sns.barplot(x='city', y='salary', data=df)
plt.show()
```

Visualizations help you understand your data better.

8 Statistical Analysis

Perform common statistical operations.

```
# Correlation matrix
df.corr(numeric_only=True)

# Compute mean, median, std
df[['age', 'salary']].agg(['mean', 'median', 'std'])

# t-test (compare two groups)
from scipy import stats
group1 = df[df['city'] == 'Bangalore']['salary']
group2 = df[df['city'] == 'Delhi']['salary']
stats.ttest_ind(group1, group2, equal_var=False)
```

Useful for data analysis and hypothesis testing.

9 Save Analysis Results

Save your cleaned data or analysis results.

```
# Save to CSV
df.to_csv('cleaned_data.csv', index=False)

# Save to Excel
df.to_excel('results.xlsx', index=False)

# Save plot
plt.savefig('plot.png')
```

Save your work for reports, dashboards or further analysis.

10 Quick Tips

- ☐ Use pip install in terminal or !pip install in notebooks.
- ☐ Always clean your data before analysis.
- ☐ Use visualizations to find patterns and trends.
- ☐ Practice with real datasets (Kaggle, UCI, etc.).
- ☐ Learn useful shortcuts (e.g., df.groupby(), df.plot()).
- ☐ Keep this cheat handy for quick reference!

Small code snippets can save you a lot of time!

Happy Analyzing!

30 Python Functions & Techniques

Every Data Scientist Should Know

Practical. Common. Time-Saving. Use these functions to write cleaner code and analyze data more efficiently.

1 print()

Display output.

```
print('Hello, World!')
```

Output:
Hello, World!

2 type()

Check the type of a variable.

```
type(3.14)
```

Output:
<class 'float'>

3 len()

Get the length of an object.

```
len([1, 2, 3, 4])
```

Output:
4

4 range()

Generate a range of numbers.

```
list(range(5))
```

Output:
[0, 1, 2, 3, 4]

5 list()

Convert to a list.

```
list((1, 2, 3))
```

Output:
[1, 2, 3]

6 tuple()

Create a tuple.

```
tuple([1, 2, 3])
```

Output:
(1, 2, 3)

7 set()

Create a set (unique values).

```
set([1, 2, 2, 3])
```

Output:
{1, 2, 3}

8 dict()

Create a dictionary.

```
dict(name='Aqsa', age=25)
```

Output:
{'name': 'Aqsa', 'age': 25}

9 sorted()

Return a sorted list.

```
sorted([3, 1, 4, 2])
```

Output:
[1, 2, 3, 4]

10 sum()

Calculate the sum.

```
sum([10, 20, 30])
```

Output:
60

11 min()

Find the minimum value.

```
min([5, 2, 9])
```

Output:
2

12 max()

Find the maximum value.

```
max([5, 2, 9])
```

Output:
9

13 round()

Round a number.

```
round(3.14159, 2)
```

Output:
3.14

14 abs()

Get the absolute value.

```
abs(-10)
```

Output:
10

15 pow()

Calculate a power.

```
pow(2, 3)
```

Output:
8

16 divmod()

Get quotient and remainder.

```
divmod(10, 3)
```

Output:
(3, 1)

17 enumerate()

Get the index and value.

```
list(enumerate(['a', 'b', 'c']))
```

Output:
[(0, 'a'), (1, 'b'), (2, 'c')]

18 zip()

Pair elements from multiple iterables.

```
list(zip([1, 2], ['a', 'b']))
```

Output:
[(1, 'a'), (2, 'b')]

19 map()

Apply a function to each item.

```
list(map(lambda x: x*2, [1,2,3]))
```

Output:
[2, 4, 6]

20 filter()

Filter items based on a condition.

```
list(filter(lambda x: x > 2, [1, 2, 3, 4]))
```

Output:
[3, 4]

21 any()

Check if any element is True.

```
any([False, True, False])
```

Output:
True

22 all()

Check if all elements are True.

```
all([True, True, False])
```

Output:
False

23 isinstance()

Check if an object is an instance of a class.

```
isinstance(5, int)
```

Output:
True

24 int()

Convert to integer.

```
int(3.99)
```

Output:
3

25 float()

Convert to float.

```
float('3.14')
```

Output:
3.14

26 str()

Convert to string.

```
str(100)
```

Output:
'100'

27 list comprehension

Create a list using a condition or expression.

```
[x for x in range(5) if x % 2 == 0]
```

Output:
[0, 2, 4]

28 open()

Open a file.

```
with open('data.txt', 'r') as f:  
    data = f.read()
```

Reads the content of a file.

29 reversed()

Return a reverse iterator of a sequence.

```
list(reversed([1, 2, 3]))
```

Output:
[3, 2, 1]

30 help()

Get help about a function.

```
help(len)
```

Shows the documentation for len().

"Master the basics. They take you a long way."

PYTHON FUNCTIONS

EVERY ML ENGINEER SHOULD KNOW

These built-in functions make data handling, model development and experimentation easier.

1 **len()**

Returns the number of items in an object

```
data = [10, 20, 30, 40]
n = len(data)
print(n) # Output: 4
```

Useful to check dataset size, number of features, etc.

2 **type()**

Returns the type of an object

```
x = [1, 2, 3]
print(type(x)) # Output: <class 'list'>
```

Helps you understand the data structure you are working with.

3 **range()**

Generates a sequence of numbers

```
for i in range(5):
    print(i)
# Output: 0 1 2 3 4
```

Commonly used in loops for iterating over data, folds, epochs, etc.

4 **enumerate()**

Returns both index and value while iterating

```
features = ['age', 'income', 'score']
for i, col in enumerate(features):
    print(i, col)
# Output:
# 0 age
# 1 income
# 2 score
```

Useful for iterating over feature names with their index.

5 **zip()**

Combines multiple iterables element-wise

```
features = ['age', 'income', 'score']
values = [25, 50000, 0.8]
for f, v in zip(features, values):
    print(f, "=", v)
# Output:
# age = 25
# income = 50000
# score = 0.8
```

Helpful for pairing features with values, predictions with labels, etc.

6 **sum()**

Returns the sum of all items in an iterable

```
values = [0.1, 0.5, 0.2, 0.2]
total = sum(values)
print(total) # Output: 1.0
```

Useful for calculating totals, losses, probabilities, etc.

7 **min() / max()**

Returns the smallest or largest item

```
scores = [0.72, 0.91, 0.68, 0.85]
print(min(scores)) # 0.68
print(max(scores)) # 0.91
```

Useful for finding best/worst performance, data range, etc.

8 **sorted()**

Returns a sorted list from an iterable

```
numbers = [3, 1, 4, 1, 5]
sorted_nums = sorted(numbers)
print(sorted_nums) # [1, 1, 3, 4, 5]
print(sorted(numbers, reverse=True))
# [5, 4, 3, 1, 1]
```

Useful for sorting metrics, feature importance, or any ordered data.

9 **round()**

Rounds a number to a given precision

```
val = 0.876543
print(round(val, 2)) # 0.88
print(round(val, 4)) # 0.8765
```

Useful for presenting metrics like accuracy, loss, etc.



Small functions, big impact – These built-in functions are simple but essential in everyday machine learning workflows. Mastering them helps you write cleaner, faster and more efficient code!

ADVANCED PYTHON FOR DATA SCIENCE



WHY ADVANCED PYTHON?

- Write efficient, readable, and scalable code
- Handle large datasets and complex workflows
- Build reusable libraries and production-ready solutions

CORE PRINCIPLES

- Pythonic code
- Modularity and reusability
- Performance and scalability

1. ADVANCED DATA STRUCTURES

defaultdict, Counter, OrderedDict

```
from collections import defaultdict, Counter
d = defaultdict(int)
d['a'] += 1
cnt = Counter(['a', 'b', 'a'])
# cnt = {'a': 2, 'b': 1}
```

Namedtuple

```
from collections import namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(2, 3)
```

2. FUNCTIONS & LAMBDAS

Lambda functions

```
square = lambda x: x**2
square(5) # 25
```

Higher-order functions

```
def apply(f, lst):
    return [f(x) for x in lst]
apply(lambda x: x+1, [1,2,3]) # [2,3,4]
```

3. ITERTOOLS

Powerful iterator building blocks

```
from itertools import combinations, groupby
list(combinations([1,2,3], 2))
# [(1,2), (1,3), (2,3)]
```

```
data = [('a',1), ('a',2), ('b',1)]
for key, group in groupby(data, key=lambda x: x[0]):
    print(key, list(group))
```

4. PANDAS ADVANCED

MultiIndex

```
import pandas as pd
arrays = [['A', 'A', 'B', 'B'], [1,2,1,2]]
index = pd.MultiIndex.from_arrays(arrays,
                                  names=['Group', 'Num'])
df = pd.DataFrame({'val': [10,20,30,40]}, index=index)
df.loc[['A',1], 'val'] # 10
```

Efficient operations

```
df.groupby(level='Group').sum()
df.pivot_table(values='val', index='Group',
               columns='Num', aggfunc='mean')
```

5. NUMPY ADVANCED

Broadcasting

```
import numpy as np
a = np.array([1,2,3])
b = np.array([10])
a + b # array([11,12,13])
```

Vectorization

```
x = np.arange(1_000_000)
%timeit np.sqrt(x)
%timeit [i**0.5 for i in x]
```

6. PERFORMANCE TOOLS

Time and profile your code

```
import time
start = time.perf_counter()
# ... your code ...
print(f"Time: {time.perf_counter() - start:.4f}s")
```

Line-by-line profiling

```
from line_profiler import LineProfiler
def func():
    # ... your code ...
lp = LineProfiler(func)
lp_wrapper = lp(func)
lp_wrapper()
lp.print_stats()
```

7. DATA VISUALIZATION

Advanced plots with Seaborn & Matplotlib

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set(style="whitegrid")
sns.boxplot(x='category', y='value', data=df)
plt.figure(figsize=(8,4))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
```

8. MULTIPROCESSING

Multiprocessing (Windows-safe)

```
from multiprocessing import Pool

def work(x):
    return x*x

if __name__ == '__main__':
    with Pool(4) as p:
        result = p.map(work, range(10))
```

9. BEST PRACTICES

- Use type hints
- Write modular functions
- Document your code
- Handle exceptions
- Use virtual environments (venv or conda)

10. MACHINE LEARNING PIPELINE

Scikit-learn pipeline example

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier())
])
pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)
acc = accuracy_score(y_test, pred)
print(f"Test Accuracy: {acc:.4f}")
```

11. WORKING WITH SQL

Using SQL in Python

```
import sqlite3
conn = sqlite3.connect('data.db')
query = "SELECT * FROM table_name LIMIT 5"
df = pd.read_sql(query, conn)
conn.close()
```

12. EXTRA TOOLS

- Jupyter for exploration
- Numba for JIT compilation
- Dask for parallel DataFrames
- Joblib for caching & parallelism

13. DECORATORS

Add functionality to functions

```
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        res = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"Time: {elapsed:.4f}s")
        return res
    return wrapper
@timer
def compute():
    sum(range(1000000))
```

14. GENERATORS

Memory-efficient iteration

```
def gen(n):
    for i in range(n):
        yield i*i
for val in gen(5):
    print(val) # 0 1 4 9 16
```

15. DATACLASSES

Simplify class creation

```
from dataclasses import dataclass
@dataclass
class Point:
    x: float
    y: float
p = Point(2.0, 3.0)
```

16. TYPE HINTS

Add type annotations

```
from typing import List, Tuple
def add(a: int, b: int) -> int:
    return a + b
def process(data: List[int]) -> Tuple[int, int]:
    return (min(data), max(data))
```

Explore
Clean
Visualize
Get Insights
Repeat !

Python EDA

"Better Data
Better Questions
Better Decisions"



20 Essential Python EDA Techniques for Real-World Data Analysis

① Import Libraries

Load the essential libraries for EDA.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

② Load Data

Load data from a CSV file.

```
df = pd.read_csv('data.csv')
df.head()
```

↳ Show first 5 rows

id	name	age	salary
1	A	25	50000
2	B	30	60000
3	C	22	45000
...	...	...	...

③ Basic Information

Understand the structure of your data.

```
df.info()
```

↳ Check data types,
non-null counts
and memory usage

```
df.shape → (rows, columns)
```

```
df.columns → column names
```

④ Summary Statistics

Get statistical summaries of numeric data.

```
df.describe()
```

	age	salary
count	1000	1000
mean	29.8	57120
std	5.6	15000
min	20	30000
25%	25	45000
50%	30	57000
75%	35	70000
max	45	100000

⑤ Check Missing Values

```
df.isnull().sum()
```

```
id      0
age     5
salary  0
department  2
experience 0
```

↳ Find missing values in each column.

⑥ Handle Missing Values

```
df['age'] = df['age'].fillna(
df['age'].mean())
```

↳ Fill missing values with mean, median or a constant value.

⑦ Check Duplicates

```
df.duplicated().sum()
```

```
3
```

```
df = df.drop_duplicates()
```

↳ Find and remove duplicate rows

⑧ Data Types

```
df.dtypes
```

```
id      int64
age     int64
salary  int64
department  object
experience int64
```

↳ Convert column data types.

```
df['salary'] = df['salary'].astype(float)
```

⑨ Unique Values

```
df['department'].unique()
```

```
array(['Sales', 'Marketing',
       'HR', 'IT', 'Finance'],
      dtype=object)
```

↳ Get unique values in a column.

⑩ Value Counts

```
df['department'].value_counts()
```

```
Sales      250
Marketing   220
IT          200
HR          180
Finance     150
```

⑪ Correlation

```
df.corr(numeric_only=True)
```

	age	salary	experience
age	1.00	0.62	0.55
salary	0.62	1.00	0.68
experience	0.55	0.68	1.00

↳ Check correlation between numeric features

⑫ Filter Data

```
high_salary = df[df['salary'] > 70000]
high_salary.head()
```

↳ Select rows based on a condition (Boolean filtering).
df[condition]

⑬ Sort Data

```
df.sort_values('salary',
ascending=False).head()
```

id	name	salary
68	A	100000
24	B	95000
17	C	90000

↳ Sort by a column.

⑭ Group By

```
df.groupby('department')['salary']
.mean()
```

```
Finance      73000.0
HR            61000.0
IT            58000.0
Marketing     52000.0
Sales        48000.0
```

↳ Perform aggregations on groups

⑮ Visualize Distribution

```
sns.histplot(df['salary'], bins=30)
plt.xlabel('Salary')
plt.ylabel('Count')
plt.title('Salary Distribution')
plt.show()
```



↳ Plot a histogram

⑯ Visualize Relationships

```
sns.scatterplot(x='experience',
y='salary', data=df)
plt.xlabel('Experience')
plt.ylabel('Salary')
plt.title('Experience vs Salary')
plt.show()
```



↳ Plot a scatter plot

⑰ Visualize Categories

```
sns.countplot(x='department',
data=df)
plt.xlabel('Department')
plt.ylabel('Count')
plt.title('Employees by Department')
plt.show()
```



⑱ Pair Plot

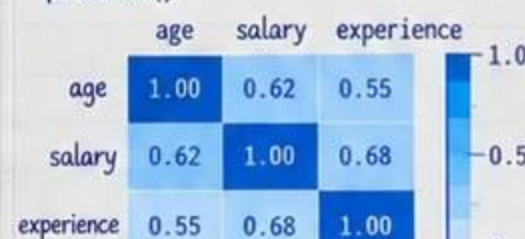
```
sns.pairplot(df[['age', 'salary',
'experience']])
plt.show()
```



↳ Visualize relationships between multiple features

⑲ Heatmap

```
sns.heatmap(df.corr(numeric_only=True),
annot=True, cmap='Blues')
plt.title('Feature Correlation')
plt.show()
```



⑳ Save Cleaned Data

```
df.to_csv('cleaned_data.csv',
index=False)
```

↳ Save the processed data to a new file.

Small Steps
Big Insights ! 😊

Keep Practicing → Better Analysis → Better Decisions



Data Today A Better Tomorrow

16 Important Python Libraries & Tools for Data Science (2026)

Better Tools → Better Insights → Smarter Decisions

1 NumPy



Fundamental package for numerical computing.

- N-dimensional arrays
- Mathematical functions
- Fast numerical computing

`pip install numpy`

2 Pandas

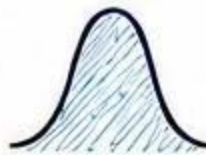


Powerful data manipulation and analysis.

- DataFrames for structured data
- Data cleaning and wrangling
- Easy analysis and summarization

`pip install pandas`

3 SciPy



Scientific computing for technical computing.

- Optimization, integration
- Linear algebra, statistics
- Signal and image processing

`pip install scipy`

4 Matplotlib



Create static, customizable plots and charts.

- Line, bar, scatter, hist plots
- Highly customizable
- Great for reports

`pip install matplotlib`

5 Seaborn



Statistical data visualization built on Matplotlib.

- Beautiful default styles
- Statistical plots
- Easy to use

`pip install seaborn`

6 scikit-learn



Simple and powerful machine learning.

- Classification, regression
- Clustering, dimensionality reduction
- Model evaluation tools

`pip install scikit-learn`

7 PyTorch



Deep learning framework built for flexibility.

- Dynamic computation & automatic differentiation
- Great for research & production
- Strong GPU support

`pip install torch`

8 Keras



High-level API for building deep learning models.

- Easy and user friendly
- Supports JAX, TensorFlow & PyTorch backends
- Build models quickly

`pip install keras`

9 XGBoost



High-performance gradient boosting library.

- Fast and efficient
- Handles large datasets
- Excellent for structured data

`pip install xgboost`

10 LightGBM



Fast, memory-efficient gradient boosting library.

- Fast training
- Low memory consumption
- Great for large-scale data

`pip install lightgbm`

11 statsmodels



Statistical modeling and econometrics.

- Regression and time series
- Hypothesis testing
- Detailed statistical results

`pip install statsmodels`

12 Plotly



Interactive and modern data visualizations.

- Interactive charts
- Web-based visualizations
- Great for dashboards

`pip install plotly`

13 Polars



Fast DataFrame processing for modern data workflows.

- Fast DataFrame processing
- Low memory usage
- Lazy execution & parallelism

`pip install polars`

14 OpenCV



Computer vision and image processing.

- Image processing
- Object detection
- Real-time computer vision

`pip install opencv-python`

15 Dask



Parallel and distributed computing for large datasets.

- Scale Pandas and NumPy workflows
- Out-of-core computation
- Work with big data

`pip install dask`

16 JupyterLab



Interactive environment for data science and research.

- Write and run code
- Visualize results instantly
- Great for learning & research

`pip install jupyterlab`

Bonus Tool: Streamlit

Build interactive data and ML web apps with Python.



- Create web apps quickly
- Great for ML demos
- Build apps without separate front-end code



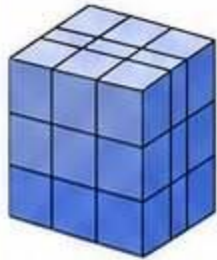
`pip install streamlit`

NUMPY CHEAT SHEET

The Foundation of Numerical Computing in Python

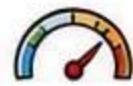
What is NumPy?

NumPy (Numerical Python) is a powerful library for working with arrays, matrices, and mathematical functions. It provides high-performance multidimensional arrays and tools for numerical computing.

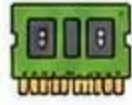


- ✓ Fast and efficient
- ✓ Often uses less memory than Python lists for numerical data.
- ✓ Easy to use
- ✓ Widely used with Pandas, SciPy, Scikit-learn, TensorFlow, and PyTorch

Why Use NumPy?



High Performance
Many operations use optimized compiled code for speed.



Memory Efficient
Often uses less memory than Python lists for numerical data.



Vectorization
Perform operations on entire arrays without loops.



Broad Functionality
Mathematical functions, linear algebra, random numbers, FFT, statistics, and more.

NumPy Array (ndarray)

A homogeneous, multi-dimensional array.

1D Array

```
arr_1d = np.array([1, 2, 3, 4])
```

```
arr_1d  
array([1, 2, 3, 4])
```

1	2	3	4
---	---	---	---

2D Array

```
arr_2d = np.array([[1, 2, 3],  
                  [4, 5, 6]])
```

```
arr_2d  
array([[1, 2, 3],  
       [4, 5, 6]])
```

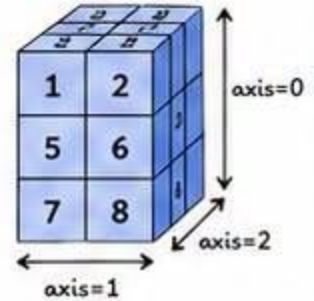
1	2	3
4	5	6

Key Attributes

3D Array

```
arr_3d = np.array([[[1, 2],  
                  [3, 4]],  
                  [[5, 6],  
                  [7, 8]]])
```

```
arr_3d  
array([[[1, 2],  
       [3, 4]],  
      [[5, 6],  
      [7, 8]]])
```



• arr.shape • arr.dtype • arr.size • arr.ndim • arr.itemsize

Creating Arrays

From List

```
np.array([1, 2, 3])  
array([1, 2, 3])
```

1	2	3
---	---	---

Zeros

```
np.zeros((2, 3))  
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

0	0	0
0	0	0

Ones

```
np.ones((2, 3))  
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

1	1	1
1	1	1

Arange

```
np.arange(0, 10, 2)  
array([0, 2, 4, 6, 8])
```

0	2	4	6	8
---	---	---	---	---

Linspace

```
np.linspace(0, 1, 5)  
array([0., 0.25, 0.5,  
       0.75, 1.])
```

0	0.25	0.5	0.75	1
---	------	-----	------	---

Identity Matrix

```
np.eye(3)  
array([[1., 0., 0.],  
       [0., 1., 0.],  
       [0., 0., 1.]])
```

1	0	0
0	1	0
0	0	1

Array Indexing & Slicing

```
arr = np.array([[1, 2, 3, 4],  
               [5, 6, 7, 8],  
               [9, 10, 11, 12]])
```

1	2	3	4
5	6	7	8
9	10	11	12

```
arr[0, 1] # 2 (element at row 0, col 1)  
arr[1]    # [5 6 7 8] (second row)  
arr[:, 2] # [3 7 11] (all rows, 3rd column)  
arr[0:2, 1:3] # [[2 3]  
                [6 7]] (slice)  
arr[-1, :] # [ 9 10 11 12] (last row)
```

Array Operations (Vectorization)

Element-wise Operations

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
a + b # [5 7 9]  
a - b # [-3 -3 -3]  
a * b # [4 10 18]  
a / b # [0.25 0.4 0.5]  
a ** 2 # [1 4 9]
```

Comparison Operations

```
a > 1 # [False True True]  
a == b # [False False False]  
a <= 2 # [ True  True  False]
```

Useful Functions

```
np.sum(a) # 6  
np.mean(a) # 2.0  
np.min(a) # 1  
np.max(a) # 3
```

Reshape & Transpose

```
arr = np.array(np.arange(1, 13))  
arr.reshape(3, 4)  
array([[ 1,  2,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])  
arr.reshape(4, 3).T  
array([[ 1,  2,  3],  
       [ 4,  5,  6],  
       [ 7,  8,  9],  
       [10, 11, 12]])
```

Universal Functions (ufuncs)

```
np.sqrt(arr) # Square root  
np.exp(arr) # e^x  
np.log(arr) # Natural log  
np.sin(arr) # Sine  
np.abs(arr) # Absolute value
```

Statistical Functions

```
np.sum(arr) # Sum  
np.mean(arr) # Mean  
np.median(arr) # Median  
np.std(arr) # Standard deviation  
np.var(arr) # Variance
```

Linear Algebra

```
A = np.array([[1, 2],  
              [3, 4]])  
B = np.array([[5, 6],  
              [7, 8]])  
A @ B # Matrix multiplication  
np.linalg.inv(A) # Inverse  
np.linalg.det(A) # Determinant
```

Random Number Generation

```
np.random.rand(2, 3) # Uniform [0,1]  
np.random.randn(2, 3) # Standard normal  
np.random.randint(1, 10, size=(2, 3))  
# Random integers  
np.random.choice([1, 2, 3], size=5)  
# Random choice
```

Broadcasting

Perform operations on arrays of different shapes.

```
a = np.array([[1], [2], [3]]) # Shape (3,1)  
b = np.array([10, 20, 30]) # Shape (3,)  
a + b # Shape (3,1)  
array([[11, 21, 31],  
       [12, 22, 32],  
       [13, 23, 33]])
```

1
2
3

10	20	30
----	----	----

11	21	31
12	22	32
13	23	33

Data Types (dtype)

```
np.int8, np.int32, np.int64 # Integers  
np.float16, np.float32, np.float64 # Floating point  
np.bool_ # Boolean  
np.complex64, np.complex128 # Complex numbers  
np.datetime64 # Date & time  
np.str_ # String
```

Check: arr.dtype Change: arr.astype(new_dtype)

Why NumPy is Important?

- ✓ Backbone of data science and machine learning.
- ✓ Used in Pandas, SciPy, Scikit-learn, TensorFlow, PyTorch.
- ✓ Speeds up computations with optimized C implementations.
- ✓ Enables efficient handling of large datasets.



NumPy = Fast Arrays + Powerful Math + Efficient Computations → Smarter Data Science

25 Pandas Operations You Actually Use

Clean Data
Explore Data
Transform Data
Get Results

Quick. Practical. Essential. A must-have cheat sheet for every data practitioner.

1 Load a CSV File

Load data from a CSV file into a DataFrame.

```
import pandas as pd
df = pd.read_csv('data.csv')
df.head()
```

id	name	age
1	A	25
2	B	30
3	C	22

2 Explore Data

Get basic information about the DataFrame.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries
Data columns (total 3 columns):
#   Column  Non-Null Count
0   id      1000 non-null
1   name    1000 non-null
2   age     950 non-null
dtypes: int64(2), object(1)
memory usage: 23.6 KB
```

3 Get Basic Statistics

Generate summary statistics for numeric columns.

```
df.describe()
```

	age	salary
count	950.0	950.0
mean	29.8	57120.0
std	5.6	15000.0
min	20.0	30000.0
25%	25.0	45000.0
50%	30.0	57000.0
75%	35.0	70000.0
max	45.0	100000.0

4 Select Columns

Select one or more columns from the DataFrame.

```
df[['name', 'age']]
```

name	age
A	25
B	30
C	22

Select multiple columns using a list.

5 Filter Rows

Filter rows based on a condition.

```
df[df['age'] > 30]
```

id	name	age
2	B	32
5	E	35
8	H	40

Filter data based on your condition.

6 Add a New Column

Create a new column using existing data.

```
df['age_group'] =
pd.cut(df['age'],
bins=[0, 25, 35, 100],
labels=['Young', 'Adult', 'Senior'])
```

age	age_group
18	Young
28	Adult
40	Senior

7 Handle Missing Values

Fill missing values in a column.

```
df['age'] = df['age'].fillna(
df['age'].mean())
```

age
25.0
30.0
29.8

Fill missing values with mean, median or a constant value.

8 Drop Missing Values

Remove rows with missing values.

```
df.dropna()
```

id	name	age
1	A	25
2	B	30
3	C	22

Remove rows that contain missing values.

9 Check for Duplicates

Find duplicate rows in the DataFrame.

```
df.duplicated().sum()
3
```

```
df.drop_duplicates()
```

id	name	age
1	A	25
2	B	30
3	C	22

Find and remove duplicate rows.

10 Sort Data

Sort the DataFrame by a column.

```
df.sort_values('salary',
ascending=False).head()
```

id	name	salary
3	C	90000
1	A	85000
2	B	75000

Sort in ascending or descending order.

11 Group By

Group data and apply an aggregation.

```
df.groupby('department')
['salary'].mean()
```

department	salary
Finance	73000.0
HR	61000.0
IT	58000.0
Marketing	52000.0

12 Value Counts

Count the occurrences of unique values.

```
df['department'].value_counts()
```

department	count
Sales	250
Marketing	220
IT	200
HR	180
Finance	150

13 Unique Values

Get unique values in a column.

```
df['department'].unique()
```

```
array(['Sales', 'Marketing',
'HR', 'IT', 'Finance'],
dtype=object)
```

Find all unique values in a column.

14 Apply a Function

Apply a function to each element in a column.

```
df['name_length'] =
df['name'].apply(len)
```

name	name_length
A	1
Abhi	4
Riya	4

Apply custom functions to columns.

15 Merge DataFrames

Combine two DataFrames using a common column.

```
pd.merge(df1, df2,
on='id', how='inner')
```

id	name	salary
1	A	50000
2	B	60000
3	C	45000

Merge using inner, outer, left or right join.

16 Concatenate Data

Stack DataFrames vertically or horizontally.

```
pd.concat([df1, df2],
ignore_index=True)
```

id	name	age
1	A	25
2	B	30
3	C	22

Combine datasets easily.

17 Pivot Table

Create a pivot table for summarizing data.

```
df.pivot_table(values='salary',
index='department',
aggfunc='mean')
```

department	salary
Finance	73000.0
HR	61000.0
IT	58000.0

Summarize data in a new format.

18 Rename Columns

Rename columns for better readability.

```
df.rename(columns={
'old_name': 'new_name'
})
```

new_name	age
A	25
B	30
C	22

Rename columns for clarity.

19 Change Data Types

Convert a column to a different data type.

```
df['salary'] =
df['salary'].astype(float)
```

```
df.dtypes
```

id	int64
name	object
age	int64
salary	float64

Convert column data types.

20 String Operations

Perform string operations on a column.

```
df['name'].str.upper()
```

name
A
ABHI
RIYA

Use string methods like upper, lower, contains, etc.

21 Date & Time

Work with date and time in pandas.

```
df['date'] = pd.to_datetime(df['date'])
df['date'].dt.year
```

date	year
2023-01-15	2023
2024-06-20	2024
2025-03-10	2025

Extract useful information from dates.

22 Conditional Column

Create a column based on a condition.

```
import numpy as np
df['bonus'] = np.where(
df['salary'] > 70000,
'Yes',
'No'
)
```

salary	bonus
80000	Yes
50000	No

23 Sample Data

Randomly select rows from the DataFrame.

```
df.sample(3, random_state=42)
```

id	name	age
8	H	28
2	B	35
5	E	31

Randomly sample rows for quick inspection.

24 Set Index

Set a column as the index of the DataFrame.

```
df = df.set_index('id')
```

name	age
1 A	25
2 B	30
3 C	22

Set a column as the index.

25 Save to CSV

Save the cleaned data to a new file.

```
df.to_csv('cleaned_data.csv',
index=False)
```



Save the processed data for future use.

Practice these. Use them. Build real skills.

Better Data → Better Insights → Better Decisions

SciPy

Scientific Computing in Python

- ☐ Easy to use
- ☐ Open-source
- ☐ Built for scientific computing
- ☐ Works with NumPy, Matplotlib and other libraries
- ☐ Widely used in research and industry

1. What is SciPy?

- SciPy is an open-source library for scientific and technical computing in Python.
- It extends NumPy with advanced scientific computing tools.
- Provides modules for optimization, integration, interpolation, linear algebra, signal processing, statistics and more.

SciPy provides powerful tools for scientific computing!

2. Installation

```
# Install using pip
pip install scipy

# Import
import numpy as np
import scipy
import scipy.linalg as la
```

Works seamlessly with Python and NumPy.

3. SciPy Modules

<code>scipy.optimize</code>	Optimization algorithms
<code>scipy.integrate</code>	Numerical integration
<code>scipy.interpolate</code>	Interpolation and curve fitting
<code>scipy.linalg</code>	Linear algebra routines
<code>scipy.signal</code>	Signal processing
<code>scipy.stats</code>	Statistical distributions and tests
<code>scipy.spatial</code>	Spatial data structures and algorithms

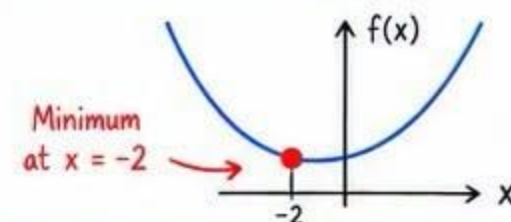
4. Optimization

Find the minimum of a function

```
from scipy.optimize import minimize

def f(x):
    return x**2 + 4*x + 4

result = minimize(f, x0=0)
print(result.x) # [-2.]
print(result.fun) # 0.0
```

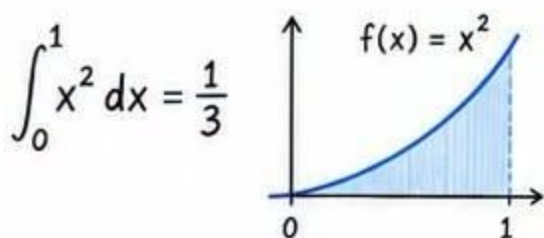


5. Integration

Compute definite integrals

```
from scipy import integrate

f = lambda x: x**2
result, error = integrate.quad(f, 0, 1)
print(result) # 0.3333
print(error) # 3.7e-15
```

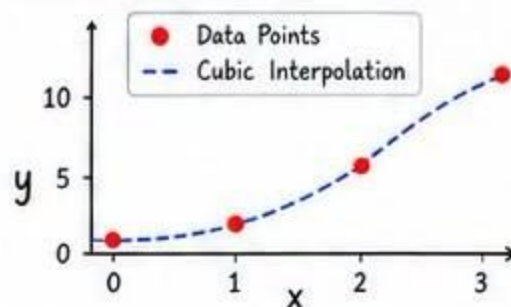


6. Interpolation

Estimate values between data points

```
from scipy.interpolate import interp1d

x = [0, 1, 2, 3]
y = [0, 1, 4, 9]
f = interp1d(x, y, kind='cubic')
print(f(1.5)) # 2.25
```



7. Linear Algebra

Matrix operations using SciPy

```
from scipy.linalg import inv, solve

A = np.array([[3, 1],
              [1, 2]])
b = np.array([9, 8])
x = solve(A, b)
print(x) # [2. 3.]

A_inv = inv(A)
print(A_inv)
```

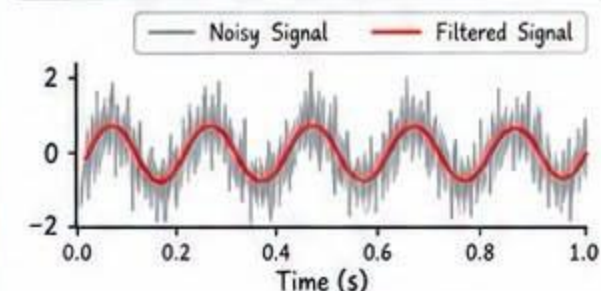
SciPy provides efficient routines for matrix operations, decompositions, and solving linear systems.

8. Signal Processing

Filter a noisy signal

```
from scipy import signal
import matplotlib.pyplot as plt

t = np.linspace(0, 1, 500)
x = np.sin(2*np.pi*5*t) + 0.5*np.random.randn(500)
b, a = signal.butter(4, 0.1)
y = signal.filtfilt(b, a, x)
```

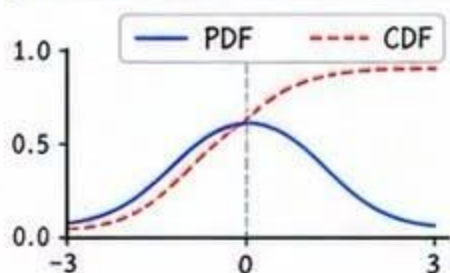


9. Statistics

Work with probability distributions

```
from scipy import stats

# Normal distribution
x = np.linspace(-3, 3, 100)
pdf = stats.norm.pdf(x, 0, 1)
cdf = stats.norm.cdf(x, 0, 1)
print(pdf[:5])
```

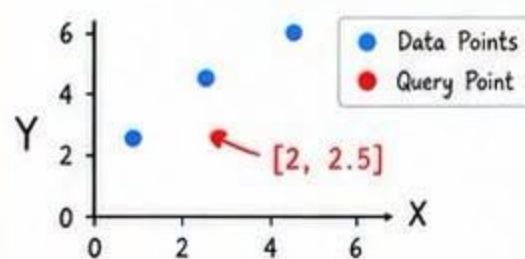


10. Spatial

Work with spatial data (e.g., nearest neighbors)

```
from scipy.spatial import KDTree

points = np.array([[1, 2], [3, 4], [5, 6]])
tree = KDTree(points)
dist, idx = tree.query([2, 2.5])
print(idx) # 0 (nearest point)
print(dist) # 1.118...
```



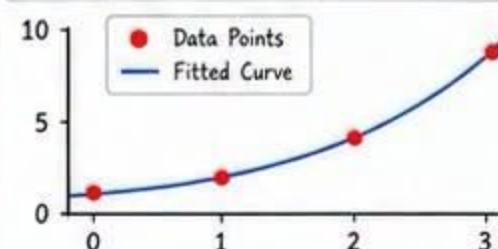
11. Real-World Example

Fit a curve to data using SciPy

```
from scipy.optimize import curve_fit

def func(x, a, b):
    return a * np.exp(b * x)

x = np.array([0, 1, 2, 3])
y = np.array([1.0, 2.1, 4.1, 8.1])
params, _ = curve_fit(func, x, y)
print(params) # [1.046 0.682]
```



12. Why Use SciPy?

- ☒ Powerful tools for scientific computing
- ☒ Well-tested and reliable
- ☒ Works seamlessly with NumPy, Matplotlib and other libraries
- ☒ Used in research, industry and academia
- ☒ Open-source and actively maintained

SciPy helps you solve real-world scientific and engineering problems with the power of Python.

SciPy = Scientific Tools + Real-World Solutions + A Better Tomorrow

Matplotlib Visualization Guide

1. Line Plot

Show trends over time or continuous data.

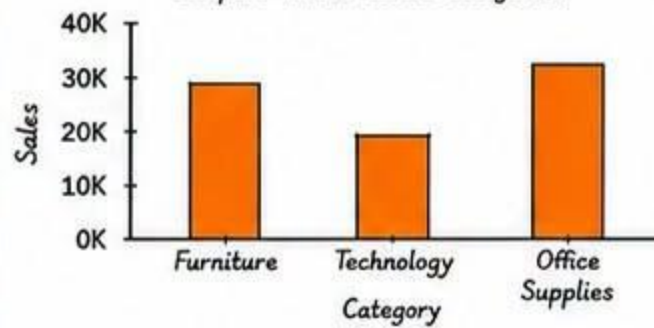


```
plt.plot(months, sales, marker='o', color='royalblue')
```

- Best for: Trends, changes over time
- Example: Monthly sales trend

2. Bar Chart

Compare values across categories.

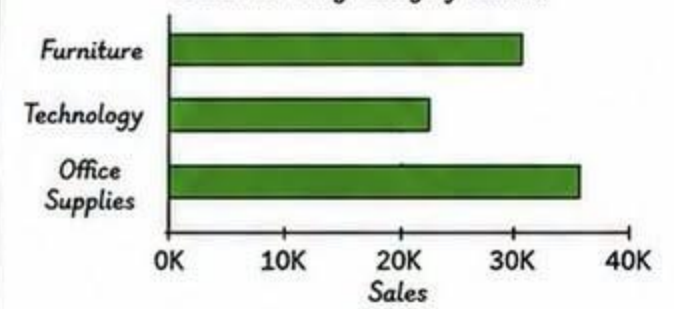


```
plt.bar(categories, sales, color='orange')
```

- Best for: Comparing categories
- Example: Sales by category

3. Horizontal Bar Chart

Useful for long category names.



```
plt.barh(categories, sales, color='seagreen')
```

- Best for: Comparing categories (long labels)
- Example: Sales by category

4. Histogram

Show distribution of a single numeric variable.

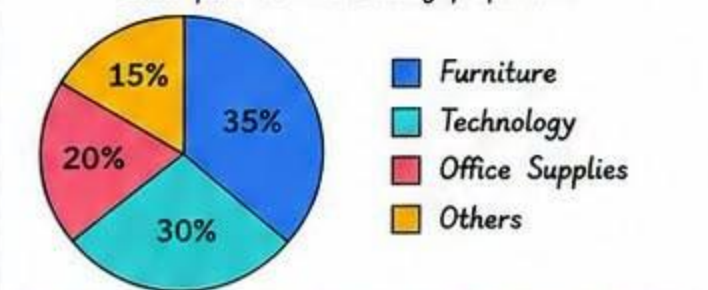


```
plt.hist(sales, bins=10, color='purple', edgecolor='black')
```

- Best for: Distribution & frequency
- Example: Sales distribution

5. Pie Chart

Show part-to-whole using proportions.

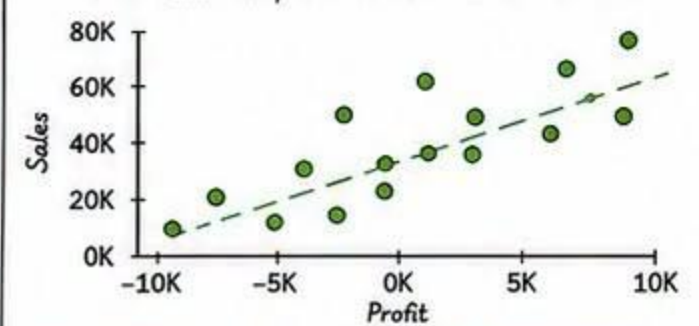


```
labels = ['Furniture', 'Technology', 'Office Supplies', 'Others']
plt.pie(sizes, labels=labels, autopct='%1.1f%%',
startangle=90, colors=['royalblue', 'teal', 'crimson', 'gold'])
```

- Best for: Proportions & composition
- Example: Sales by category

6. Scatter Plot

Show relationship between two numeric variables.

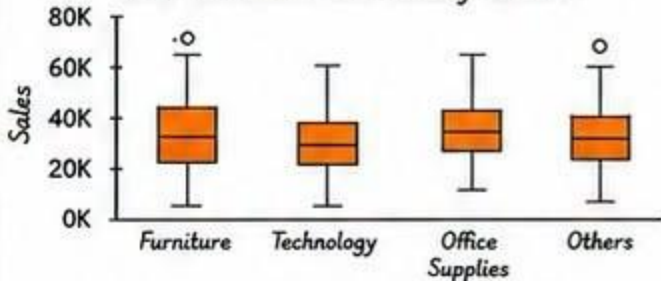


```
plt.scatter(profit, sales, color='forestgreen')
```

- Best for: Relationships & correlations
- Example: Sales vs Profit

7. Box Plot

Show distribution and identify outliers.



```
plt.boxplot(data, labels=categories, patch_artist=True,
boxprops=dict(facecolor='orange'))
```

- Best for: Distribution & outliers
- Example: Sales distribution by category

8. Area Chart

Show trends over time with a filled area.



```
plt.fill_between(months, sales, color='skyblue', alpha=0.4)
plt.plot(months, sales, color='steelblue', marker='o')
```

- Best for: Trends over time with a filled area
- Example: Monthly Sales Trend

9. Heatmap

Show patterns & intensity in a matrix.



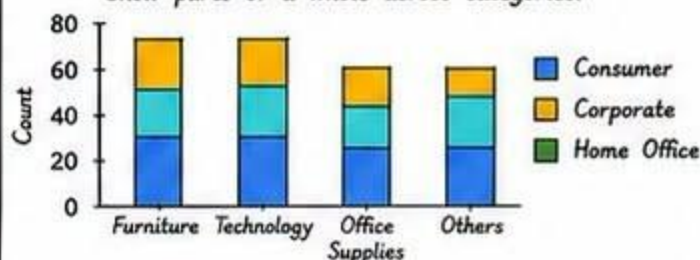
```
corr = np.array([[1.00, 0.72, -0.25],
[0.72, 1.00, -0.18],
[-0.25, -0.18, 1.00]])
```

```
plt.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)
labels = ['Sales', 'Profit', 'Discount']
plt.colorbar(); plt.xticks(range(3), labels);
plt.yticks(range(3), labels)
```

- Best for: Correlations & patterns
- Example: Correlation heatmap

10. Stacked Bar Chart

Show parts of a whole across categories.



```
plt.bar(x, consumer, label='Consumer')
plt.bar(x, corporate, bottom=consumer, label='Corporate')
plt.bar(x, home, bottom=[c+co for c,co in zip(consumer,corporate)],
label='Home Office')
plt.legend()
```

- Best for: Parts of a whole across groups
- Example: Segment count by category

11. Dual Axis Plot

Compare two measures with different scales.

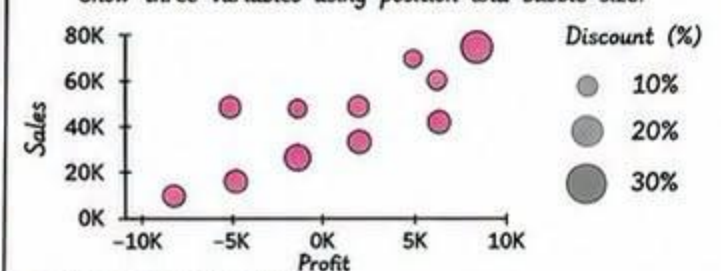


```
fig, ax1 = plt.subplots()
ax1.plot(months, sales, color='royalblue', marker='o')
ax2 = ax1.twinx()
ax2.plot(months, profit, color='darkorange', marker='s')
```

- Best for: Comparing two measures
- Example: Sales and Profit Trend

12. Bubble Chart

Show three variables using position and bubble size.



```
profit = np.array(profit)
sales = np.array(sales)
discount = np.array([10, 20, 30, 10, 20, 30, 10, 20, 30, 30])
plt.scatter(profit, sales, s=discount*100, c='deeppink',
alpha=0.6, edgecolors='black')
```

- Best for: 3 variables in one view
- Example: Sales, Profit & Discount

Tip: Choose the right plot to reveal insights and make your data story clear!

EDA

Interview Questions

Common Questions Asked in Data Science & ML Interviews

1 What is EDA?

- Exploratory Data Analysis (EDA) is the process of understanding your data before building any model.
- Helps in uncovering patterns, detecting anomalies, testing hypotheses and checking assumptions.



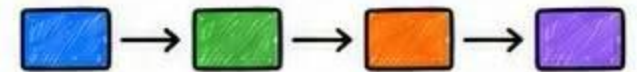
2 Why is EDA important?

- Helps understand data better.
- Detects missing values, outliers and inconsistencies.
- Provides insights for feature engineering.
- Improves model performance.



3 What are the steps in EDA?

1. Understand the data
2. Inspect data quality
3. Explore distributions
4. Analyze patterns & relationships
5. Detect outliers
6. Check distributions assumptions
7. Summarize insights



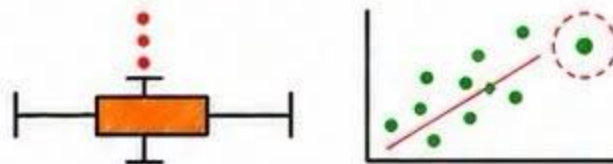
4 How do you handle missing data?

- Identify missing values using `df.isna().sum()`.
- Visualize missingness (heatmap).
- Handle using:
 - Remove rows/columns
 - Imputation (mean, median, mode)
 - Forward/Backward fill



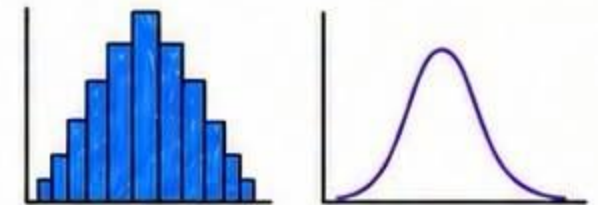
5 How do you detect outliers?

- Use box plots, scatter plots.
- Use statistical methods:
 - IQR Method
 - Z-score
 - Percentile Method



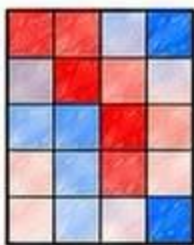
6 How do you check the distribution of a variable?

- Use histogram, KDE plot, box plot.
- Check for skewness and kurtosis.
- Consider transformations if highly skewed.



7 How do you check relationships between variables?

- Use scatter plots for two variables.
- Use correlation matrix / heatmap for multiple variables.
- Use correlation coefficient:
 - Pearson (linear)
 - Spearman (monotonic)



8 How do you deal with categorical variables?

- Analyze frequency using `value_counts()`.
- Visualize using bar plot or count plot.
- Check cardinality (number of unique values).
- Identify rare categories.



9 How do you handle multicollinearity?

- Check correlation matrix.
- Use VIF (Variance Inflation Factor).
- Remove highly correlated features.
- Combine related features.
- Use regularization if appropriate.

Feature	VIF
A	1.2
B	5.6
C	1.8
D	7.2

10 How do you check class imbalance?

- Use `value_counts()` on target variable.
- Visualize using bar plot or pie chart.
- Handle using:
 - Resampling (Over/Under)
 - SMOTE
 - Class weights



11 How do you summarize your findings?

- Use summary statistics (`df.describe()`).
- Highlight key observations.
- Support with visualizations.
- Share actionable insights.



12 What are common mistakes during EDA?

- ✗ Not understanding the data.
- ✗ Ignoring missing values or outliers.
- ✗ Relying only on summary statistics.
- ✗ Not validating assumptions.
- ✗ Skipping data visualization.



Tip: Good EDA leads to better features, better models and better insights!



All Important Functions for Exploratory Data Analysis (EDA) in Python (Pandas, NumPy, Matplotlib, Seaborn)

1 Load and View Data

<code>pd.read_csv('file.csv')</code>	Load a CSV file
<code>df.head()</code>	View first 5 rows
<code>df.tail()</code>	View last 5 rows
<code>df.shape</code>	Get number of rows and columns
<code>df.columns</code>	View column names
<code>df.info()</code>	Summary of data types and non-null counts
<code>df.describe()</code>	Summary statistics (numeric columns)

First step in EDA – get to know your data!

2 Data Types & Missing Values

<code>df.dtypes</code>	Check data types
<code>df.isnull().sum()</code>	Count missing values
<code>df.isnull().mean()*100</code>	% missing values
<code>df.dropna()</code>	Drop missing values
<code>df.fillna(value)</code>	Fill missing values
<code>df.duplicated().sum()</code>	Count duplicate rows
<code>df.drop_duplicates()</code>	Remove duplicates
<code>df.nunique()</code>	Unique values in each column

Clean data leads to better insights!

3 Basic Statistics

<code>df.mean(numeric_only=True)</code>	Mean of numeric columns
<code>df.median(numeric_only=True)</code>	Median
<code>df.std(numeric_only=True)</code>	Standard deviation
<code>df.min(numeric_only=True)</code>	Minimum values
<code>df.max(numeric_only=True)</code>	Maximum values
<code>df.quantile([0.25, 0.5, 0.75], numeric_only=True)</code>	Percentiles
<code>df['col'].value_counts()</code>	Frequency of values in a column

Understand the distribution of your data!

4 Data Selection & Filtering

<code>df['col']</code>	Select a column
<code>df[['col1', 'col2']]</code>	Select multiple columns
<code>df.iloc[row_index]</code>	Select by position
<code>df.loc[row_index]</code>	Select by label
<code>df[df['col'] > 10]</code>	Filter rows by condition
<code>df.query('col > 10')</code>	Filter using query
<code>df.sort_values('col')</code>	Sort values
<code>df.sample(5)</code>	Random sample of rows

Select and filter the data you need!

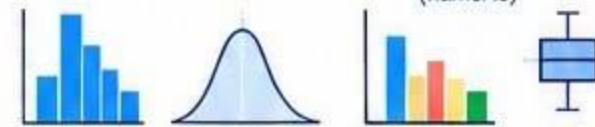
5 Working with Categorical Data

<code>df['col'].unique()</code>	Unique values
<code>df['col'].nunique()</code>	Number of unique values
<code>df['col'].value_counts()</code>	Value counts
<code>df['col'] = df['col'].astype('category')</code>	Convert to category type
<code>pd.get_dummies(df)</code>	One-hot encoding
<code>df['col'].cat.categories</code>	Categories in a column
<code>df.groupby('col').size()</code>	Count by category

Understand your categorical variables!

6 Data Visualization (Univariate)

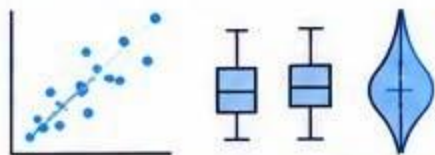
<code>df['col'].hist()</code>	Histogram (numeric)
<code>df['col'].plot(kind='kde')</code>	KDE plot
<code>df['col'].value_counts().plot(kind='bar')</code>	Bar plot (categorical)
<code>df['col'].plot(kind='density')</code>	Density plot
<code>df.boxplot(column='col')</code>	Box plot (numeric)



Visualize the distribution of each variable!

7 Data Visualization (Bivariate)

<code>df.plot(x='x', y='y', kind='scatter')</code>	Scatter plot
<code>sns.scatterplot(data=df, x='x', y='y')</code>	
<code>df.groupby('col')['num'].mean().plot(kind='bar')</code>	Group by plot
<code>sns.boxplot(x='cat', y='num', data=df)</code>	Box plot
<code>sns.violinplot(x='cat', y='num', data=df)</code>	Violin plot
<code>sns.stripplot(x='cat', y='num', data=df)</code>	Strip plot
<code>sns.pairplot(df)</code>	Pairplot (all numeric)



Select and relationships between variables!

8 Correlation & Relationships

<code>df.corr(numeric_only=True)</code>	Correlation matrix
<code>np.corrcoef(x, y)</code>	Correlation coefficient
<code>sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')</code>	
<code>sns.pairplot(df)</code>	Pairwise relationships

	A	B	C	D
A	1.00	0.75	0.20	-0.10
B	0.75	1.00	0.30	-0.25
C	0.20	0.30	1.00	0.45
D	-0.10	-0.25	0.45	1.00

Find how variables are related!

9 Outliers & Distribution Checks

<code>sns.boxplot(y=df['col'])</code>	Box plot for outliers (numeric)
<code>sns.boxplot(x='cat', y='value', data=df)</code>	Outliers by category
<code>df['col'].quantile([0.25, 0.75])</code>	IQR calculation

$IQR = Q3 - Q1$

Lower bound = $Q1 - 1.5 \times IQR$

Upper bound = $Q3 + 1.5 \times IQR$



Identify and handle outliers!

10 Date & Time Functions

<code>pd.to_datetime(df['date'])</code>	Convert to datetime
<code>df['date'].dt.year</code>	Extract year
<code>df['date'].dt.month</code>	Extract month
<code>df['date'].dt.day</code>	Extract day
<code>df['date'].dt.dayofweek</code>	Day of week (0=Mon)
<code>df.set_index('date')</code>	Set date as index
<code>df.resample('ME').sum()</code>	Resample (e.g. monthly)

Useful for time series and trend analysis!

11 Useful NumPy Functions

<code>np.array([1, 2, 3])</code>	Create an array
<code>np.mean(a)</code>	Mean
<code>np.median(a)</code>	Median
<code>np.std(a)</code>	Standard deviation
<code>np.min(a), np.max(a)</code>	Min, Max
<code>np.percentile(a, [25, 50, 75])</code>	Percentiles
<code>np.corrcoef(x, y)</code>	Correlation coefficient
<code>np.where(condition)</code>	Conditional operation

Helpful for numerical computations!

12 Quick Tips for EDA

- 1 Start with `shape`, `info()` and `describe()`.
- 2 Check and handle missing values.
- 3 Understand data types and convert if needed.
- 4 Analyze both numeric and categorical variables.
- 5 Use visualizations to find patterns and outliers.
- 6 Look for correlations and relationships.
- 7 Try `groupby()` for deeper insights.
- 8 Document your findings!
- 9 A clean EDA makes better models!

Same Data. Different Questions. Better Answers!

EDA Workflow

Exploratory Data Analysis Workflow

① Dataset Overview

ID	Age	Salary	Gender	City
1	25	50000	M	Delhi
2	32	62000	F	Mumbai
3	28	54000	M	Bengaluru
...	...	...	...	...
10000	45	75000	F	Pune

- Rows: 10,000
- Columns: 15
- Sample records shown above

Dataset Overview

② Shape Check

Shape = (10000, 15)

- Rows: 10,000
- Columns: 15

Shape Check

③ Data Types

Column	Dtype
Age	int64
Salary	int64
Gender	object
Date	datetime64[ns]
...	...

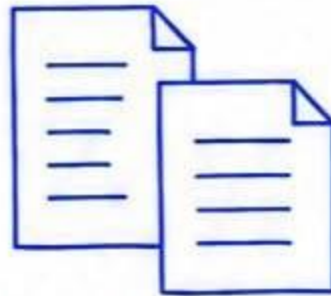
Data Types

④ Missing Values

Column	Missing Count	Missing %
Age	120	1.20%
Salary	200	2.00%
City	150	1.50%
Gender	0	0.00%
...	...	...

Missing Values

⑤ Duplicate Rows



Duplicate rows found:
35

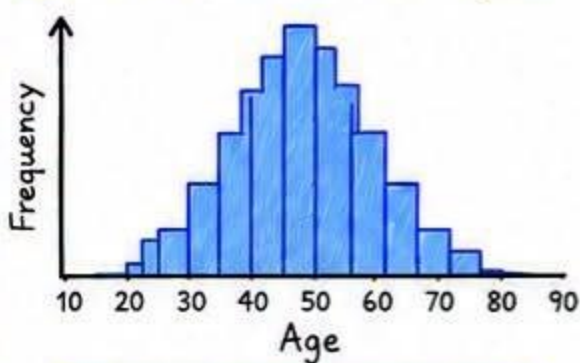
Duplicate Rows

⑥ Summary Statistics

Metric	Age	Salary
Count	10000	10000
Mean	45.2	60321.1
Std	12.4	15023.8
Min	18	21000
25%	36	50000
55%	44	60000
75%	54	70000
Max	82	125000

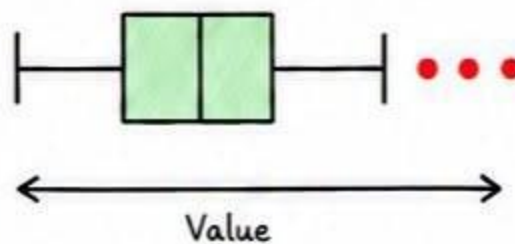
Summary Statistics

⑦ Distribution Analysis



Distribution Analysis

⑧ Outlier Detection



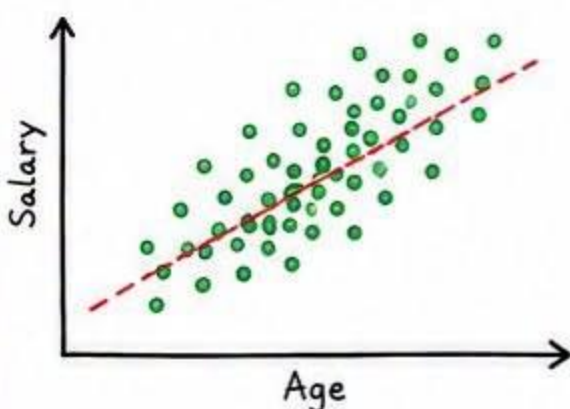
Outlier Detection

⑨ Correlation Analysis

	A	B	C	D
A	1.00	0.80	0.20	-0.10
B	0.80	1.00	0.50	0.00
C	0.20	0.50	1.00	-0.30
D	-0.10	0.00	-0.30	1.00

Correlation Analysis

⑩ Feature Relationships



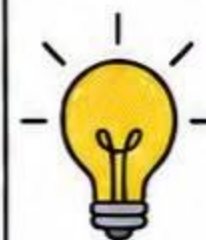
Feature Relationships

⑪ Class Distribution



Class Distribution

⑫ Insights & Next Steps



- ✓ Missing values identified
- ✓ Outliers detected
- ✓ Key relationships found
- ✓ Data quality checked



Ready for
Feature Engineering

Insights & Next Steps

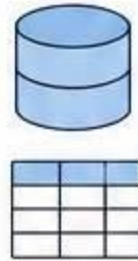
TYPES OF DATA ANALYTICS

Different types of data analytics help us understand data, make better decisions, and drive future outcomes.

1. DESCRIPTIVE ANALYTICS

- Summarizes historical data
- Answers: What happened?
- Uses aggregation, reporting, dashboards, and visualization
- Helps track performance and identify patterns

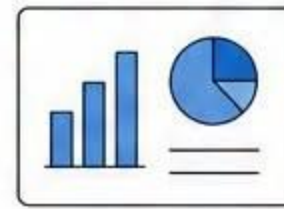
Raw Data



Process



Summary / Insights



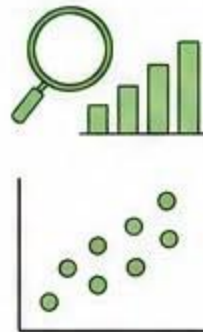
Example

Sales by region last quarter was \$2.3M

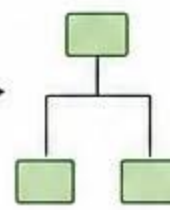
2. DIAGNOSTIC ANALYTICS

- Examines data in detail to identify patterns, relationships, and likely root causes
- Answers: Why did it happen?
- Uses drill-down, correlations, filters, and root cause analysis
- Helps understand what influenced the results

Data Exploration



Analysis



Findings

✓ High discounts are a likely contributing factor to lower profit in Q2.

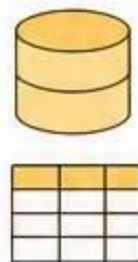
Example

Analysis suggests high discounts contributed to lower profit in Q2.

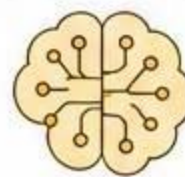
3. PREDICTIVE ANALYTICS

- Uses historical data to predict future outcomes
- Answers: What is likely to happen?
- Uses statistical models and machine learning to predict future outcomes
- Helps in risk assessment, demand forecasting, etc.

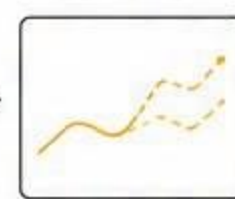
Historical Data



Predictive Model



Prediction



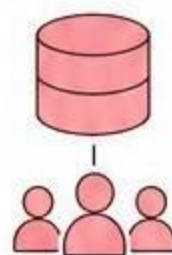
Example

Next month sales are expected to be \$2.7M.

4. PRESCRIPTIVE ANALYTICS

- Recommends actions to achieve the best outcome
- Answers: What should we do?
- Uses optimization, simulation, business rules, and ML
- Helps in decision-making and strategy planning

Data + Context



Analysis & Optimization



Recommended Actions



Example

Increase ad spend in Region A to maximize profit.

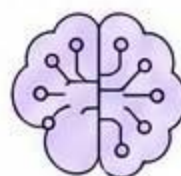
5. COGNITIVE ANALYTICS

- Uses AI, NLP, and machine learning to use AI techniques to generate insights from complex data
- Answers: What can we learn and automate?
- Uses deep learning, NLP, knowledge graphs, etc.
- Helps in advanced insights, automation, and AI-driven decisions

Unstructured & Structured Data



AI Models



Insights



Example

AI analyzes customer reviews to identify key themes, trends, and actionable insights.

Key Takeaways

Each type of analytics builds on the previous one and serves a unique purpose. Together, they turn raw data into smart decisions!

- Descriptive → What happened?
- Diagnostic → Why did it happen?
- Predictive → What is likely to happen?
- Prescriptive → What should we do?
- Cognitive → What can we learn and automate?

All Important Formulas for Data Analytics

① Descriptive Statistics

Mean (Average)	$\bar{x} = \frac{\sum x_i}{n}$
Median	Middle value (after sorting)
Mode	Most frequent value
Variance (Sample)	$s^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1}$
Standard Deviation	$s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n - 1}}$
Range	Max - Min
Interquartile Range (IQR)	$Q_3 - Q_1$

Helps summarize and understand your data!

② Probability

Probability of an event (when outcomes are equally likely)	$P(A) = \frac{\text{Number of favorable outcomes}}{\text{Total outcomes}}$
Complement rule	$P(A^c) = 1 - P(A)$
Conditional probability	$P(A B) = \frac{P(A \cap B)}{P(B)}$
Bayes' theorem	$P(A B) = \frac{P(B A)P(A)}{P(B)}$

Useful for A/B testing, user behavior and risk analysis.

③ Correlation & Relationship

Pearson correlation coefficient	$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$
---------------------------------	--

r ranges from -1 to 1
(1 = strong positive, -1 = strong negative)

Helps measure the strength and direction of a relationship.

④ Linear Regression

Simple linear regression equation	$y = \beta_0 + \beta_1 x$
Slope (β_1)	$\beta_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$
Intercept (β_0)	$\beta_0 = \bar{y} - \beta_1 \bar{x}$
R-squared	$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

Used to predict a continuous outcome.

⑤ Hypothesis Testing

Z-test statistic	$Z = \frac{\bar{x} - \mu}{\sigma / \sqrt{n}}$ (population standard deviation known)
------------------	--

T-test statistic	$t = \frac{\bar{x} - \mu}{s / \sqrt{n}}$
------------------	--

Chi-square statistic	$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$
----------------------	---

p-value	Probability of observing the result (if the null hypothesis is true)
---------	--

Helps you make data-driven decisions.

⑥ Time Series

Moving Average (MA)	$MA_t = \frac{\sum_{i=0}^{k-1} x_{t-i}}{k}$
Exponential Moving Average (EMA)	$EMA_t = \alpha x_t + (1 - \alpha) EMA_{t-1}$
Growth Rate	$\frac{\text{New} - \text{Old}}{\text{Old}} \times 100$
Compound Annual Growth Rate (CAGR)	$\left[\left(\frac{\text{Ending Value}}{\text{Beginning Value}} \right)^{\frac{1}{n}} - 1 \right] \times 100$ n = number of years (or periods)

Useful for trend analysis and forecasting.

⑦ Business & Marketing Metrics

Conversion Rate	$\frac{\text{Total Conversions}}{\text{Total Visitors}} \times 100$
-----------------	---

Customer Retention Rate	$\frac{\text{Ending Customers} - \text{New Customers}}{\text{Starting Customers}} \times 100$
-------------------------	---

Churn Rate	$\frac{\text{Customers Lost}}{\text{Total Customers}} \times 100$
------------	---

Simplified Customer Lifetime Value (CLV)	$CLV = \text{Average Purchase Value} \times \text{Purchase Frequency} \times \text{Customer Lifespan}$
--	--

Common metrics in data analytics roles.

⑧ Data Cleaning & Outliers

Z-score (for outliers)	$z = \frac{x - \bar{x}}{s}$
IQR method	Lower bound = $Q_1 - 1.5 \times IQR$ Upper bound = $Q_3 + 1.5 \times IQR$
Min-Max Scaling	$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$
Standardization (Z-score scaling)	$x' = \frac{x - \bar{x}}{s}$

Helps prepare clean and consistent data.

⑨ Common Business Calculations

Percentage Change	$\frac{\text{New} - \text{Old}}{\text{Old}} \times 100$
-------------------	---

Weighted Average	$\bar{x}_w = \frac{\sum w_i x_i}{\sum w_i}$
------------------	---

Absolute Change	$ \text{New} - \text{Old} $
-----------------	-----------------------------

Relative Change	$\frac{\text{New} - \text{Old}}{\text{Old}}$
-----------------	--

Useful for reports, dashboards and performance analysis.

Quick Tips:

- ☐ Understand what each formula means (not just memorize).
- ☐ Know when and where to use it.
- ☐ Practice with real datasets (Excel, SQL, Python, etc.).
- ☐ Focus on the interpretation of results.
- ☐ Keep this sheet for quick reference!